

第二章 软件生命周期模型

期末复习讲义 · 软件工程 · 开卷考试速查 · A4 双栏打印版

本章速览

核心概念：

- 软件工程过程 4 基本活动（规格说明 / 开发 / 确认 / 演进）
- 软件生命周期 6 基本活动（计划 P / 需求 D / 设计 D / 编码 D / 测试 C / 维护 A）
- 戴明环 PDCA 与软件工程过程的对应
- 软件过程模型 3 类型：工作流 / 数据流 / 角色-动作

核心问题：

- 各模型的"是什么 / 优缺点 / 适用场景"
- 需求明确 → 瀑布；需求模糊 → 演化/原型；需求模糊+核心清楚 → 增量；大型高风险 → 螺旋；OO+迭代 → 喷泉/RUP；面向对象+用例驱动 → RUP；需求频繁变 → 敏捷/XP

高频考点：

- 瀑布模型优缺点（"推迟软件实现"、文档驱动、缺乏灵活性）
- 原型方法 3 类型 + 2 策略（废弃 / 追加）
- RUP 二维模型（4 阶段 + 9 工作流 + 3 大特点）
- 敏捷宣言 4 价值观；XP 测试（单元+验收）
- 螺旋模型 = 风险驱动
- 增量 vs 演化的本质区别

必备对比：

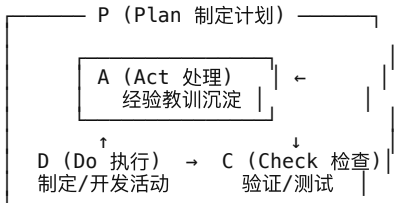
- 瀑布 / 演化 / 增量 / 螺旋 4 模型对比
- V / W 模型 vs 瀑布
- RUP 4 阶段 4 里程碑

题目关键词导航（看到这些词 → 立即定位）

- **高频** "推迟软件实现" → 瀑布模型优点第 2 条
- **高频** "风险驱动 / 风险分析" → 螺旋模型（非成本控制！）
- **高频** "用例为驱动 + 架构为核心 + 迭代增量" → RUP 三大特点
- **高频** "面向对象驱动 / 阶段重叠无间隙" → 喷泉模型
- **高频** "只有一次原型开发" → 瀑布（无迭代）
- "不是完整的方法论" → 敏捷建模（AM）
- "需求不明确" → 演化模型 / 原型方法
- "需求模糊 + 核心需求清楚" → 增量模型
- "用户故事 US / 配对编程 / TDD" → XP 极限编程
- "生命周期从概念到废弃" → 不要选"从需求分析到上线"
- "原型方法可用于其他阶段" → ✓ 正确
- "V/W 解决需求不明确" → × 错误（仍只适合需求稳定）

知识主线

图 2-1 戴明环 PDCA（软件工程过程的 4 抽象活动）



对应到生命周期：P=制定计划；D=需求/设计/编码；C=测试；A=运行维护

全章主线：

1. 软件工程 = 过程 + 方法 + 工具，过程是骨架
2. 戴明环 PDCA 是所有工程过程的抽象原型
3. 软件过程 4 活动 → 生命周期 6 活动
4. 生命周期需要"模型"来组织 → 软件生命周期模型
5. 传统模型 9 种：瀑布、演化、增量、喷泉、螺旋、V/W、构件组装、RAD、原型
6. 新型模型 2 大类：RUP（统一过程）+ 敏捷（XP/Scrum 等）
7. RUP = 喷泉 + 增量 + 用例驱动 + 架构为核心
8. 敏捷 = 4 价值观 + 短迭代 + 用户全程参与

核心知识点详解（PPT 内容主导）

1. 软件工程过程

工程项目 3 目标：**合理进度、有限经费、一定质量**。戴明（Deming）针对质量目标提出 **PDCA 循环（戴明环）**：Plan / Do / Check / Act。

软件工程过程定义：为获得软件产品，在软件工具支持下由软件工程师完成的一系列软件工程活动。包含 4 项基本活动：

1. **软件规格说明**（Specification）：规定软件的功能及使用限制
2. **软件开发**（Development）：产生满足规格说明的软件
3. **软件确认**（Validation）：通过有效性验证保证满足客户要求
4. **软件演进**（Evolution）：为满足变更要求在使用中不断改进

2. 模型及软件生命周期定义

模型：实际事物/系统的抽象化表示，抽取主要因素，忽略次要因素。模型省略细节，便于操作。

软件过程模型 3 类型（按过程 3 组成成分：活动/工件/角色）：

- **工作流（Work Flow）模型**：强调**活动**的序列、输入输出、依赖关系（如瀑布）
- **数据流（Data Flow）模型**：强调**工件变换**关系（输入→输出）
- **角色/动作模型**：强调**参与角色**与职能域划分

软件生命周期：软件产品从考虑其概念开始到不再使用为止的整个时期，包括概念、分析与设计、构造、移交和运行等阶段。

3. 软件生命周期 6 基本活动（必背）

序号	活动	PDCA	核心产出
1	制定计划	P	可行性研究报告 + 实施计划
2	需求分析	D	需求规格说明书 + 用户手册初稿
3	软件设计	D	概要设计 + 详细设计说明书
4	程序编码	D	源程序清单（结构良好）
5	软件测试	C	单元/组装/确认（有效性）测试报告
6	运行维护	A	改正性/适应性/完善性维护

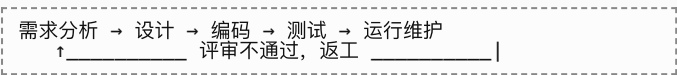
维护 3 类型：改正性（修错误） / 适应性（适应环境变化） / 完善性（增强功能）。维护是最复杂的开发活动。

4. 瀑布模型 (Winston Royce, 1970) ★ 必考

定义：规定软件生命周期中的基本工程活动，并规定它们 自上而下、相互衔接的固定次序，如同瀑布流水逐级下落。

4 特征（每阶段必做）：

- 1. 工作对象来自上一活动的输出（里程碑文档，须"冻结"）
- 2. 按本阶段活动规程执行任务
- 3. 产出本阶段软件工件，作为下一活动输入
- 4. 对本阶段执行情况进行评审（不通过则"解冻"返工）



优点 3 条：

- 1. 阶段划分降低复杂度，提高透明度与可管理性
- 2. 推迟软件实现，强调编码前必须完成分析与设计（避免急于编码导致结构混乱）
- 3. 以阶段评审与文档控制为手段指导全过程，及时发现缺陷保证质量

缺点 3 条：因此只适用于需求明确的软件项目

- 1. 缺乏灵活性，无法解决需求不明确或不准确（最突出缺点）
- 2. 风险控制能力弱（成品只有测试后可见；体系结构级风险只有组装测试才能发现）
- 3. 文档驱动，文档过多增加工作量；以文档完成度评估进度会产生错误结论

适用：需求明确的中小型项目，特别适合军工/嵌入式/合规性强的系统。

"do twice": Royce 原论文在分析之前加入"初步程序设计"，被视为演化模型基础。瀑布模型：是所有其他软件生命周期模型的基础。

5. 演化模型 ★ 易混 进化模型

背景：瀑布模型需求难一次明确 → 提倡两次开发。第一次试验开发得到原型（探索可行性，弄清需求）；第二次在此基础上获得满意产品。

分类（按原型应用策略）：

- 探索式：与用户共同工作，从清楚的部分开始，逐步添加功能
- 抛弃式：原型只为明确需求，需求明确后原型废弃，重新开发正式系统

优点：明确用户需求；提高系统质量；降低开发风险；可在开发中间向用户展示半成品。

缺点：难于管理；结构较差；可能丧失瀑布的文档控制；可能导致系统结构较差；技术可能不成熟。

适用：需求不清楚；小型或中小型系统；开发周期短。不适用：大型、长生命周期项目。

6. 增量模型 (Mills 等, 1980) ★ 必考

定义：首先对系统最核心或最清晰的需求进行分析、设计、实现、测试并集成；再按优先级逐步对后续需求进行上述工作，逐步建成完整系统。结合瀑布模型和演化模型的优点。

核心思想：核心需求优先；以稳定的体系结构指导所有增量集成。

优点 4 条：

- 1. 客户可在第一次增量后使用核心功能，增强信心
- 2. 项目总体失败风险低（核心功能即使某增量失败仍可用）
- 3. 最高优先级功能得到最多次测试，保障重要功能可靠性
- 4. 所有增量在同一体系结构指导下集成，提高稳定性和可维护性

缺点 2 条：

- 1. 增量粒度难以选择
- 2. 确定所有的基本业务服务比较困难

典型例：字处理软件 4 增量 = {文件管理/编辑/文档生成} → {更完善编辑/文档生成} → {拼写文法检查} → {高级页面布局}。

7. 喷泉模型 (迭代模型) ★ 面向对象

定义：也称迭代模型。认为软件开发各阶段是相互重叠和多次反复的，像喷泉：水喷上去又落下，可落在中间或底部。各阶段无特定次序，可并行，可在任何阶段补充其他阶段遗漏的需求。

2 大特征：

- 迭代性：演化、增量、螺旋也都体现
- 无间隙性：后一阶段能自然复用前一阶段成果（设计复用分析、实现复用设计）

本质：以面向对象驱动模型（对象在分析/设计/实现阶段表达方式统一）。

优点：提高开发效率；缩短开发周期。

缺点：难于管理（阶段重叠致工件与开发人员管理困难）。

8. 螺旋模型 (Boehm, 1988) ★ 风险驱动

针对：大型软件项目开发周期长、风险高。

最大特点 = 风险驱动（注意：不是"成本控制"！纵轴的累计成本只是绘图坐标）

4 象限活动（每个螺旋回路都经历）：

象限	活动	核心任务
①	制定计划	确定目标、约束、实施方案、可行性
②	风险分析	识别风险、评估概率/频率/损害、制定规避措施（核心特征）
③	实施工程	执行本开发阶段活动（可结合原型/瀑布）
④	客户评估	客户反馈 + 评审，决定是否进入下一回路

适用：大型软件开发。吸收演化模型的"演化"概念，要求开发人员和客户对每个演化中出现的风险进行分析并规避。局限：风险分析需丰富评估经验，应用难度大。

9. V 模型与 W 模型

背景：瀑布将测试作为编码之后的独立阶段 → 分析设计的错误纠正时机大为推迟。1980 年代后期 Paul Rook 提出 V 模型，将测试提前；Evolutif 公司提出 W 模型。

V 模型：左半部分 = 测试前的瀑布（开发阶段）；右半部分 = 测试阶段。明确划分测试级别并对应开发阶段：

测试级别	对应开发阶段	主要目的
单元测试	详细设计	发现编码错误，验证与详细设计一致
集成测试	概要设计	检查单元间接口缺陷
系统测试	系统/软件需求分析	检查功能/性能是否达标
验收测试	用户需求获取	确认满足用户需求/合同

V 模型局限：保留瀑布缺点（测试作为独立阶段），并未提高抗风险能力。

W 模型：由两个 V 型组成（测试 V + 开发 V），测试与开发同步进行，测试对象不仅是程序，也包括需求和设计（扩充 V&V 即确认 Validation 和验证 Verification）。仍只适合需求稳定的项目。

10. 构件组装模型

定义：利用模块化思想，复用构件库中的软件构件，通过组装高效率、高质量构造软件系统。本质上是演化的，开发过程是迭代的。维护 = 构件升级/替换/扩充。

5 阶段：需求定义和分析 → 体系结构设计（关键） → 构件开发 → 应用软件构造 → 测试与发布。

优点：充分利用复用提高效率；允许多项目同时开发；降低费用；可分步提交。

缺点：缺乏通用构件标准（风险大）；可重用性与高效性难协调；过分依赖构件质量。

11. 快速应用开发 RAD

定义：增量型软件开发过程模型，强调极短的开发周期（如 60~90 天）。使用**构件组装方法**快速开发，使用 4GL（第四代语言）。

5 阶段：业务建模 → 数据建模 → 过程建模 → 应用生成（复用/创建构件） → 测试及迭代。

适用：管理信息系统（MIS）；可模块化、每个主要功能模块可在 3 个月内完成的项目。

缺陷：并非所有应用都适合（不能模块化或技术风险高/外围互操作高的不适合）；时间紧沟通不当易失败。

12. 原型方法（Prototyping）★ 高频考点

背景：瀑布需精确需求但实际很难一次得到完整准确规格说明。原因：①用户开始只有模糊想法；②开发推进后用户产生新要求；③开发者遇到设计/实现困难需改变需求。

原型定义：模拟某种最终产品的原始模型。原型方法 = 获得一组基本需求后，通过快速分析构造出小型软件系统原型，满足用户基本要求；用户试用后反馈，减少误解。

3 类型（按目的）：

类型	目的	典型场景
探索型	弄清用户要求，确定所期望特性，探讨多种方案可行性	需求模糊、双方对项目都缺乏经验
实验型	考核技术方案是否合适、分析设计规格说明是否可靠	大规模开发之前
进化型	适应需求变化，逐步将原型进化成最终系统	需求经常变动

- 2 策略：**
- 废弃策略：原型用完即弃；探索型、实验型属此类
 - 追加策略：原型作为核心，逐步追加演化为最终系统；进化型属此类

原型方法（续）：应用过程与适用范围

应用过程 5 步：快速分析 → 快速构造 → 用户使用 → 评价反馈 → 修改（迭代）。

可支持生命周期 5 种范围：

- 辅助或代替分析阶段
- 辅助设计阶段
- 代替分析与设计阶段
- 代替分析、设计和实现阶段
- 代替全部开发阶段（典型演化模型）

优点：良好沟通手段；增进需求理解；提供学习手段；容易确定性能/可行性；最终版本可成为产品组成部分。

局限性：①大型系统不先整体划分直接用原型很困难；②大量运算/逻辑性强的模块难构造原型；③业务流程混乱的项目构造困难；④批处理系统不适用。

存在问题：文档易忽略；建立原型工作可能浪费；项目难规划管理。

13. RUP（Rational Unified Process）★ 必考

定义：由 Rational 公司（现 IBM）开发的面向对象程序开发方法论。既是软件生命周期模型，又是支持 OO 开发的工具。

3 大特点（必背）：以用例为驱动，以软件体系结构为核心，应用迭代及增量的新型软件生命周期模型。

二维结构：

- 横轴（时间维）：4 阶段 + 多次迭代 → 动态组织
- 纵轴（内容维）：9 个核心工作流 → 静态结构

本质：融合喷泉模型（迭代）和增量模型（增量）的综合生命周期模型；通过多次迭代将瀑布一个完整的开发过程分解成多个小的瀑布过程。

RUP 4 阶段 4 里程碑（必背）

阶段	目标	里程碑	关键产出
初始 Inception	了解业务，确定项目边界、规模、验收规范；识别关键用例	软件目标里程碑	愿景、原始用例模型、原始业务风险、原型
细化 Elaboration	分析问题领域，建立体系结构基础，编制项目计划，完成高风险关键需求	体系结构里程碑	风险分析文档、体系结构基线、项目计划、可执行进化原型
构造 Construction	将所有剩余技术构件和稳定业务功能开发并集成，所有功能详细测试（“制造过程”）	运行能力里程碑	可运行软件产品（beta 版）、用户手册
交付 Transition	确保软件对最终用户可用，跨越多次迭代；产品测试 + 基于反馈的少量调整	产品发布里程碑	最终软件产品包、安装介质、培训/支持手册

每阶段结尾执行评估，满足目标才能进入下一阶段。

RUP 9 核心工作流（必背）

- 6 核心过程工作流**（类似瀑布的几个阶段，但在迭代中实施）：
- 业务建模 Business Modeling
 - 需求 Requirements
 - 分析和设计 Analysis & Design
 - 实现 Implementation
 - 测试 Test
 - 部署 Deployment
- 3 核心支持工作流：**
- 配置和变更管理 Configuration & Change Management
 - 项目管理 Project Management
 - 环境 Environment

RUP 最佳实践

- 短时间分区式迭代（2~4 周，不鼓励延迟）
- 适应性开发：小步骤、快速反馈和调整
- 在早期迭代中解决高技术风险和高业务价值问题，建立内聚的核心架构
- 不断验证质量：尽早、经常、实际地测试
- 使用用例驱动软件建模（获取需求/计划/设计/测试/文档的驱动力量）
- 可视化软件建模：使用 UML
- 仔细地管理需求（拙劣的需求管理是项目陷入麻烦的常见原因）
- 实行变更请求和配置管理

14. 敏捷建模 (Agile Modeling, AM)

起源：Scott W. Ambler 总结；2001 年敏捷联盟成立；主要特点 = 快速及灵活地响应变更。
重要：AM 不是完整的方法论，只是已有生命周期模型的补充（具体应用：XP / Scrum / Crystal / FDD / ASD / DSDM / RUP）。

敏捷宣言 4 价值观（高频考点，必背）：

左（更看重）	右（仍有价值）
个体与交互 Individuals & interactions	过程与工具 Processes & tools
可工作的软件 Working software	面面俱到的文档 Comprehensive documentation
客户合作 Customer collaboration	合同谈判 Contract negotiation
响应变化 Responding to change	遵循计划 Following a plan

AM 5 价值观：沟通、简单、反馈、勇气、谦逊。

15. 极限编程 XP (Kent Beck, 1996) ★ 必考

定义：Kent Beck 在 Daimler Chrysler 项目中引入，轻量级软件开发方法，以实践为基础的软件工程过程和思想。强调用户满意，对需求变化快速反应。

四大准则：沟通、简单、反馈、勇气（后被敏捷联盟作为核心准则）。

适用：需求经常变化或风险较高的项目；小项目（2~12 人）；新领域或新技术。

工作流程：以 User Stories（US 用户故事）为驱动 → 体系结构探索 → 评估优先级 → 发布/交付计划 → 迭代开发（含迭代计划）→ 版本发布 → 验收测试 → 修复 → 新版本。

注意：XP 用 用户故事 (US) 描述需求，不是 UML 用例。

XP 关键实践：

- 配对编程 Pair Programming（代码所有权归于整个开发队伍 Collective Code Ownership）
- 测试驱动开发 TDD（先写单元测试，再写程序）
- 持续集成 Continuous Integration（每次整合都运行单元测试）
- 严格编程规范、设计复核、代码复核、重构（Refactory）
- 每周 40 小时不提倡加班；每日站立短会；开放式工作环境

XP 测试 2 大类（按主体）：①单元测试（开发人员写，TDD）②验收测试/功能测试（用户写）。还包括整合、负荷、系统测试。所有测试是 XP 最重要的文档之一，也是最终交付内容之一。

极限编程为什么强调“小步快跑”？快速交付、及时反馈、降低风险

补充知识点（教材独有 / 加深理解）

软件生命周期的内涵

- 对开发人员：从构思到交付的一系列活动和过程
 - 对用户/维护人员：从安装开始，经过长时间使用、功能改进、版本升级，直到淘汰
- 生命周期长短取决于软件质量、灵活性和适应能力；越长说明软件价值越高，开发维护成本越低。

构件组装模型 5 阶段

需求定义和分析 → 软件体系结构设计（关键）→ 构件开发 → 应用软件构造 → 测试和发布。其中体系结构设计规划迭代开发的构件数量和次序。

RAD 模型 5 阶段

- 业务建模（捕获信息流：什么信息驱动？生成什么？谁生成？流向何处？）
- 数据建模（细化数据对象、标识特征、定义关系）
- 过程建模（详细定义业务处理功能）
- 应用生成（用 4GL / 复用构件 / 自动生成）
- 测试及迭代（已有构件已测过，新构件必须测试，接口完全测试）

原型方法 5 应用过程

- 快速分析：分析员+用户配合，确定初步需求
- 快速构造：使用可视化工具/4GL；可忽略健壮性/异常处理等细节
- 用户使用：用户在开发者指导下试用
- 评价反馈：沟通发现的问题，消除误解
- 修改：制定下一版方案，迭代

选择原型的影响因素（Boar, 1984）

因素	适合原型	不适合
系统结构	联机事务、相互关联应用	批处理系统
逻辑结构	有结构的（运行支持、MIS）	大量算法和逻辑结构
用户特征	不满足预定义、愿投资、愿参与	—
应用约束	—	在线运行的补充

敏捷建模 5 价值观 + 核心原则

5 价值观：沟通、简单、反馈、勇气、谦逊。

核心原则：主张简单、拥抱变化、软件开发的第二目标是可持续性、递增的变化、令客户收益最大化、有目的地建模、多种模型、高质量工作、快速反馈、软件产品是主要目标、轻装前进。

补充原则：内容比表示更重要、三人行必有我师、了解建模方法与工具、局部调整、开放诚实沟通、利用直觉。

XP 的角色与职责

角色	主要职责
用户	编写 US 并定优先级；讨论需求；编写并运行验收测试；驱动迭代/版本计划
开发人员	讨论 US 并估计时间；细化为编码任务；编写单元测试；编码；重构；整合测试
项目经理	对外联系、组织团队、获取资源、管理团队
项目跟踪人员	跟踪版本计划、迭代计划、验收测试
教练/指导员	顾问指导；监督；帮助解决问题；可参加配对编程

RUP 裁剪 5 步

- 确定本项目需要的工作流
- 确定每个工作流需要的工件
- 确定 4 阶段之间的演进计划（以风险控制为原则）
- 确定每个阶段内的迭代计划
- 规划工作流内部结构（用活动图）

软件过程模型 vs 软件生命周期模型

软件过程模型：从一个特定角度提出的对软件过程的简化描述，是开发实际过程的抽象。包括构成软件过程的各种活动（Activities）、软件工件（artifacts）以及参与角色（Actors/Roles）等。
软件生命周期模型：描述从需求定义到废弃为止跨越整个生存期的全部过程、活动和任务的**结构框架**，同时描述不同阶段产生的软件工件，明确活动执行角色。是指导开发人员按确定框架进行开发的**标准**。

核心对比表

表 1 4 大传统模型横向对比（必考）

维度	瀑布	演化	增量	螺旋
提出者	Royce 1970	—	Mills 1980	Boehm 1988
需求	必须明确	不明确	框架+逐步明确	变化大/不确定
核心特征	文档驱动、阶段评审	两次开发/原型迭代	按优先级分增量交付	风险驱动/4 象限
灵活性	差	较好	较好	好
风险控制	弱	较强	较强	极强（核心特征）
适用规模	中小型	小型/中小型	中大型	大型
关键缺点	无法应对需求变更	结构差、难管理	粒度难选	需丰富风险评估经验

表 2 增量 vs 演化（高频混淆点）

对比项	增量模型	演化模型
需求框架	开始时已知整体框架+优先级	需求高度模糊
核心需求	清晰	不一定清晰
体系结构	开始即确立，所有增量在其指导下集成	逐步演化，可能结构较差
交付方式	按优先级分批交付，每批加新功能	原型基础上多次迭代
典型场景	"需求模糊但核心需求清楚"	"需求完全不清楚"

表 3 瀑布 / V / W 模型对比

对比项	瀑布	V 模型	W 模型
测试时机	编码后独立阶段	提前到编码后但分级	贯穿整个生命周期
测试对象	程序	程序（分 4 级对应开发阶段）	需求/设计/程序（V&V）
测试与开发关系	串行	串行（仅测试分级别）	并行（双 V 同步）
解决需求不明确？	否	否	否（仍需需求稳定）
抗风险能力	弱	未提高	有所提高

表 4 RUP 4 阶段速记

阶段	关键词	里程碑	风险
初始 Inception	边界、愿景、关键用例	软件目标	业务风险
细化 Elaboration	体系结构基线、解决高风险	体系结构	技术风险（在此阶段全部解决）
构造 Construction	制造过程、剩余功能集成测试	运行能力（beta 版）	进度/成本
交付 Transition	产品发布、用户培训、少量调整	产品发布	市场/部署

口诀：初软目，细体建，构运能，交产发。

表 5 RUP vs 瀑布（风险控制对比）

对比项	瀑布	RUP
开发组织	线性单次	迭代多次（每次小瀑布）
需求处理	前期一次性冻结	迭代中逐步稳定，初始+细化阶段重点
风险识别时机	仅测试阶段发现	每次迭代开始时风险分析
体系结构风险	组装测试才发现	细化阶段已建立体系结构基线
用户反馈	仅在交付后	每次迭代后获得
市场风险	高（一次性上市）	低（可适时发布中间版本）

表 6 原型方法 3×2 矩阵

类型 \ 策略	废弃策略	追加策略
探索型	√	—
实验型	√	—
进化型	—	√

表 7 各模型对应需求明确性

需求状态	首选模型
完全明确且稳定	瀑布、V、W
完全不清楚	演化模型、探索型原型
核心清楚，细节不清	增量模型
需求多变（大型项目）	螺旋模型
面向对象+迭代	喷泉模型、RUP
需求频繁变化（小团队）	敏捷 / XP
管理信息系统 MIS	RAD

开放题答题模板

模板 1：为什么瀑布模型要“推迟软件实现”？

答题要点：

- "推迟"含义：在编码（软件实现）之前必须先完成需求分析和软件设计。
- 对比：早期小作坊式开发拿到项目就急于编码，导致代码频繁/重复改动，结构混乱，工作效率下降，质量受损。
- 瀑布强制先分析后设计后编码，分析与设计文档作为阶段里程碑评审、冻结后才能进入下一阶段。
- 意义：降低软件复杂度；提高过程透明度和可管理性；以阶段评审和文档控制保证质量；避免开发人员因急于编码忽视分析设计。

教材 2.3.1 节瀑布模型优点第 2 条。

模板 2：螺旋模型如何控制风险？

答题步骤：

- 定义风险 = 软件开发中可能损害过程和产品质量的潜在不利因素（项目规模越大越复杂，风险越大）。
- 螺旋模型每个回路都有 4 象限，其中"风险分析"象限是其区别于其他模型的最大特征。
- 风险分析具体做法：明确每个项目风险 → 估计概率、频率、损害程度 → 制定风险管理措施规避（如：需求不清晰就开发原型；可靠度要求高就原型试验技术方案；时间性能要求高就原型试验算法）。
- 风险驱动 + 客户评估 = 在风险造成危害之前及时识别、分析、采取策略，减少甚至消除风险损害。
- 大型项目应用难度：需丰富评估经验与深厚专业知识。

模板 3：需求不明确的项目应选择哪种模型？

答题要点（按情况区分）：

1. **需求完全不清楚、小型系统** → **演化模型**（或探索型原型方法）：通过原型迭代探索需求。
2. **需求模糊但核心需求清楚** → **增量模型**：先开发核心功能的增量，后续根据反馈补充。
3. **需求多变且大型项目** → **螺旋模型**：每次迭代都有风险分析，规避需求变化风险。
4. **面向对象项目+需求变化** → **喷泉模型 / RUP**：阶段重叠迭代，用例驱动+架构为核心。
5. **需求频繁变化且团队小型** → **敏捷 / XP**：短迭代（2~4 周）、用户全程参与、拥抱变化。

模板 4：分析增量模型在演化模型基础上的改进

答题要点：

1. 演化模型虽然能处理需求不明确，但导致**系统体系结构混乱、管理不透明、丧失瀑布文档控制优点**。
2. 增量模型的改进：
 3. ① 引入**需求框架和优先级**划分，核心需求在首个增量实现；
 4. ② **先确定体系结构**，所有增量在其指导下集成，保证结构稳定；
 5. ③ 不同增量可根据内容特点选择瀑布或演化模型，**结合瀑布和演化优点**；
 6. ④ 客户可尽早使用核心功能，**降低项目总体失败风险**；
 7. ⑤ 高优先级功能获得多次测试，保障重要功能可靠性。

模板 5：原型方法在软件生命周期中的应用

答题要点：

1. 原型方法定义 + 3 类型（探索型/实验型/进化型）+ 2 策略（废弃/追加）。
2. 应用过程 5 步：快速分析 → 快速构造 → 用户使用 → 评价反馈 → 修改。
3. 可应用阶段（5 种范围）：
 4. ① 辅助或代替分析阶段；② 辅助设计阶段；③ 代替分析与设计阶段；④ 代替分析、设计和实现阶段；⑤ **代替全部开发阶段（典型演化模型）**。
5. 局限性：大型系统需先整体划分；大量运算模块不适用；流程混乱项目不适用；批处理系统不适用。

模板 6：RUP 的二维模型与三大特点

答题要点：

1. **二维结构**：横轴（时间）= 4 阶段（初始/细化/构造/交付）+ 多次迭代；纵轴（内容）= 9 核心工作流（6 核心 + 3 支持）。
2. 每个阶段结束于一个主要里程碑；每次迭代都包含需求/设计/实施/部署/测试等活动。
3. **3 大特点**：用例为驱动、软件体系结构为核心、应用迭代及增量。
4. **本质**：融合喷泉模型（迭代）和增量模型（增量）的综合模型；每个迭代相当于一个微型瀑布。
5. **裁剪**：RUP 是通用模板，具体项目须裁剪 5 步。

模板 7：XP 中几种测试？

答题要点：

1. **按测试主体 2 大类**：
 2. ① **单元测试**（开发人员写）：TDD 先写测试，每次整合后运行，代码复核/修改时运行，发现 Bug 增加测试；
 3. ② **验收测试/功能测试**（用户写）：用户为每个 US 指定验收测试，每个迭代结束时运行，通过后发布新版本。
4. **按测试层级**：单元测试、整合测试、功能测试、负荷测试、系统测试。
5. **地位**：所有测试是 XP 最重要的文档之一，也是最终交付给用户的内容之一。

历年真题汇总与详解

一、判断题（共 18 题）

2008—2009 期末 螺旋模型最大的特点是加入了对软件成本的控制。（）

答案：×

解析：螺旋模型（Boehm, 1988）最大的特点是**风险驱动**，每个回路都包含“风险分析”象限。教材 2.3.5 节明确将“风险分析”列为 4 象限活动之一，并指出瀑布、演化、增量模型“都没有考虑对这些风险因素进行分析”。纵轴的“累计成本”只是绘制坐标，并非主要特征。

2009—2010 期末 敏捷模型只能适用于面向对象技术的软件开发。（）

答案：×（错误）

解析：敏捷模型强调迭代增量交付、用户紧密协作、快速响应变化、轻量文档与过程，其核心价值观（敏捷联盟 4 条价值观）**并不限定实现技术**。教材 2.4.2 节列举的敏捷过程主要特点包括“能够适应需求的不断变化”、“快速交付可运行的软件版本”等，与是否采用面向对象技术无关；面向对象只是常见的实现手段之一。

2009—2010 期末 螺旋模型最大的特点是加入了对软件成本的控制。（）

答案：×（错误）

解析：同上。螺旋模型最大特点是**风险驱动**——每个迭代周期加入风险分析环节。教材 2.3.5 节强调其“每个螺旋周期都要进行风险识别、风险分析和风险消除”，而非成本控制。

2010—2011 期中 除了分析阶段用于明确需求以外，原型方法也可以用于软件工程的其他阶段。（）

答案：√

解析：教材 2.3.9 节明确指出“软件原型方法是在分析阶段为了明确需求而研究的方法和技术中产生的，但它也可面向软件开发的其他阶段，比如用在概要设计阶段以选择不同的软件体系结构，用在详细设计阶段以试验不同算法的实现性能等”。同节还列举了原型方法可支持生命周期的 5 种不同范围。

2010—2011 期中 原型方法既可以同瀑布模型结合也可以与 UP 模型结合使用。（）

答案：√

解析：教材 2.3.9 节列出原型可介入分析、设计、分析+设计、分析+设计+实现乃至“代替全部开发阶段”（典型演化模型）等多种范围；2.3.1 节也提到 Winston Royce 在瀑布模型基础上引入“初步程序设计”阶段即被视为演化模型的基础。原型作为一种通用思想，可与瀑布、演化、增量、RUP/UP 等模型组合使用。

2010—2011 期中 软件生命周期模型，是从一个特定角度提出的对软件过程的简化描述，是对软件开发实际过程的抽象。（）

答案：√

解析：教材 2.2.1 节对软件过程模型的定义与本题表述完全一致：“软件过程模型是从一个特定角度提出的对软件开发过程的简化描述或称之为框架描述，是对软件开发实际过程的抽象。”

2012-2013 期末 瀑布模型是所有其他软件生命周期模型的基础。()

答案: ✓

解析: 教材 2.3.6 节指出 V 模型"左半部分就是在测试阶段之前的瀑布模型"; 2.3.4 节指出"螺旋模型包含软件开发其他的模型.....当需求明确时, 螺旋模型又可能包括瀑布模型"; 2.3.9 节指出原型方法是在分析阶段研究瀑布缺陷的产物。瀑布模型规定了软件生命周期阶段划分与文档驱动的基本框架, V/W、螺旋、增量、原型等方法均针对瀑布某一缺陷改进但保留了其阶段性框架, 故可视为基础。

2013-2014 期末 极限编程与 UP 模型也是基于瀑布模型演化而来。()

答案: ✓

解析: 教材 2.3.1 节指出 Winston Royce 最初提出的瀑布模型已包含"二次开发"思想, 是演化模型的基础; 2.3.4 节明确"统一过程模型 (RUP) 本质上就是喷泉模型, 较好地结合增量模型和瀑布模型的优点"; 2.4.1 节描述 RUP 通过多次迭代将"一个完整的开发过程分解成多个小的瀑布过程"。从历史演化与活动组成角度看, UP 与 XP 都被视为在瀑布模型基础上的演化/改进。

2013-2014 期末 时至今日软件生命周期模型已成为软件快速开发的障碍。()

答案: ×

解析: 教材 2.2.2、2.3 节明确指出: 软件生命周期模型是"指导软件开发人员按照确定的框架结构和活动进行软件开发的标准", 其作用是降低开发复杂度、提高透明度与可管理性。**模型本身不是障碍**; 相反, 新型模型 (如 XP、RUP、增量、喷泉) 正是为支持快速开发而提出。

2013-2014 期末 选择瀑布模型后用户的需求不得在后期开发过程中修改。()

答案: ×

解析: 教材 2.3.1 节指出瀑布模型"缺乏灵活性, 特别是无法解决软件需求不明确或不准确的问题", 并描述了通过 **"冻结—解冻—返工"** 机制处理后期变更 (特征 4, 图 2-3 具有返工过程的瀑布模型)。即需求变更被严格管控但 **并非绝对禁止**; 规范做法是通过阶段评审与文档控制进行返工。

2015-2016 期中 除了分析阶段用于明确需求以外, 原型方法也可以用于软件工程的其他阶段。()

答案: Y (正确)

解析: 教材 2.3.9 节明确指出: "软件原型方法是在分析阶段为了明确需求而研究的方法和技术中产生的, 但它也可面向软件开发的其他阶段, 比如用在概要设计阶段以选择不同的软件体系结构, 用在详细设计阶段以试验不同算法的实现性能等。" 同节还列举了原型方法可支持生命周期 5 种不同范围。

2015-2016 期中 敏捷思想的主要目的是缩短开发周期使得用户无法变更需求。()

答案: N (错误)

解析: 敏捷思想的核心是 **"拥抱变化"**——通过短周期迭代快速交付并主动响应用户的需求变化, **而非"使得用户无法变更需求"**。教材 2.3 节开头将"以敏捷思想为代表的新型软件生命周期模型"与瀑布模型对比, 正是为了克服瀑布模型"一旦需求固定就很难修改"的缺点。本题表述与敏捷思想背道而驰。

2018-2019 期末 UP 模型是一种以用例为驱动并融合了喷泉和增量模型的软件生命周期模型。()

答案: ✓

解析: 教材 2.4.1 节指出 RUP "用例为驱动, 以架构为核心的迭代增量式开发模型"。喷泉模型亦称迭代模型 (2.3.4 节"喷泉模型也称为迭代模型"), 因此"融合喷泉和增量"等价于"迭代+增量", 与 RUP 特点相符。

2023-2024 期末 极限编程的功能需求是通过用例的方式进行描述。()

答案: ×

解析: 极限编程 (XP) 使用 **用户故事 (User Story, US)** 来描述需求, **而不是 UML 用例 (Use Case)**。用户故事是简短的、写在卡片上的一句话需求描述, 与正式的用例 (包含用例图、用例描述、系统顺序图、操作契约) 相比要轻量得多。

2024-2025 期中 原型方法是一种生命周期模型不能应用于其它模型中。()

答案: ×

解析: 教材 2.3.9 节"原型方法支持的软件生命周期"明确指出, 原型方法既可以辅助或代替分析阶段, 也可以辅助设计阶段, 甚至代替全部开发阶段 (即典型演化模型)。它可与瀑布、螺旋等模型组合使用, 并非"不能应用于其它模型"。

2024-2025 期中 V 模型并不能解决瀑布模型无法解决需求不明确的问题。()

答案: ✓

解析: 教材 2.3.6 节明确: V 模型只是在测试阶段对瀑布模型进行改动, 强调测试分级与开发阶段对应, "但它保留了瀑布模型的缺点, 即将测试作为一个独立的阶段, 所以并没有提高模型抵抗风险的能力"。因此 V 模型并未解决需求不明确的问题 (这一问题由原型、演化、增量、螺旋等模型应对)。

2024-2025 期中 极限编程中也需要进行必要的设计活动才可以进行编码实现。()

答案: ✓

解析: 教材 2.4.3 节列出 XP 工作流程时明确: "XP 是以 User Stories (US) 为驱动并考虑相应的软件体系结构探索和确认后.....", 并在"5. 极限的设计"小节中说明 XP 包含设计复核、代码复核、重构优化等设计活动。即便是轻量级的 XP, 必要的体系结构探索与设计活动仍是编码实现的前提。

2024-2025 期中 极限编程中不需要进行验收测试就可以交付给客户使用和反馈。()

答案: ×

解析: 教材 2.4.3 节明确 XP 流程: "每个迭代之后都会进行版本的发布, 随之根据客户制订的验收测试用例进行验收测试.....直到每一个迭代提交的版本通过测试和客户确认后, 发布一个新的版本。"用户职责之一即"编写验收测试, 并运行验收测试"。**验收测试是 XP 不可或缺环节**。

二、单项选择题（共 12 题）

2008-2009 期末 下面关于软件生命周期模型描述正确的是（ ）

- A. 软件生命周期是指从软件需求分析到上线运行的全过程
- B. 原型方法只能用于软件的需求分析阶段
- C. 按照瀑布模型开发系统时，必须完成需求分析才能开始系统设计
- D. 增量模型又叫做迭代模型

答案：C

解析：A 错：软件生命周期是从概念到废弃的全过程（含运行维护），教材 2.2.2 节明确指出“从考虑其概念开始，到该软件产品不再使用为止”。B 错：原型方法可应用于生命周期 5 种范围。C 正确：瀑布模型各阶段“自上而下、相互衔接的固定次序”。D 错：教材 2.3.6 节“喷泉模型也称为迭代模型”，并非增量模型。

2009-2010 期末 下面哪一个生命周期模型只有一次原型开发（ ）

- A. UP 模型 B. 喷泉模型 C. 瀑布模型 D. 演化模型

答案：C

解析：瀑布模型按需求→设计→编码→测试→维护严格按线性顺序一次完成，没有原型迭代；“一次原型”对应它“一锤子买卖、严格串行”的特征。UP（统一过程）有多个迭代周期；喷泉模型以迭代和无间隙为特征；演化模型本身就是多次原型迭代。

2010-2011 期中 对于一个需求不明确的软件项目，应该选用下面哪种软件生命周期模型。（ ）

- A. 瀑布模型 B. V 模型 C. W 模型 D. 演化模型

答案：D

解析：瀑布 / V / W 模型均要求需求在前期明确，不适合需求不明确的项目；而演化模型正是为应对需求不明确而生。教材 2.3.2 节明确指出“演化模型主要针对需求不是很明确的软件项目，希望通过第一个原型来逐步探索和理解用户需求”。

2010-2011 期中 下面哪种生命周期模型本身不是一个完整的方法论。（ ）

- A. 螺旋模型 B. 喷泉模型 C. RUP D. 敏捷模型

答案：D（敏捷模型）（注意 2010-2011 期中 id=150 同题答案给 B 喷泉模型，存在争议，详见下题）

解析：教材 2.4.2 节明确指出“敏捷建模（Agile Modeling, AM）……它只是一种态度，不是一个说明性过程；它描述一种建模风格……AM 已有生命周期模型的补充，本身不是一个完整的方法论”。螺旋、喷泉、RUP 都有完整的阶段、活动、工件定义。标准答案应为 D。

2010-2011 期中（同题异卷） 下面哪种生命周期模型本身不是一个完整的方法论（ ）

- A. 螺旋模型 B. 喷泉模型 C. RUP D. 敏捷模型

答案：D（敏捷模型）（原题库标注为 B，但与教材 2.4.2 不符，应以 D 为准）

解析：教材 2.4.1 节明确 RUP“是一个面向对象的程序开发方法论”，是完整方法论；敏捷思想（含 XP/Scrum 等）也有完整方法论体系。螺旋模型 4 象限活动框架本身构成一种方法论。喷泉模型虽描述阶段重叠与多次反复，本身需配合其他模型使用，但从教材“AM 不是完整方法论”的明确表述来看，最严谨的答案应为 D 敏捷模型。考生若遇此题，按教材原文优先选 D。

2010-2011 期中 对于一个无法快速确定完整需求的软件项目，但其核心需求比较清楚的前提下，应该选用下面哪种软件生命周期模型（ ）

- A. 瀑布模型 B. V 模型 C. 增量模型 D. 演化模型

答案：C（增量模型）

解析：教材 2.3.3 节描述增量模型“可以让客户得到一些机会以延迟对详细需求的决策，即客户的需求可以逐步提出来”，且核心需求优先级最高、在最前面的增量实现，后续增量根据反馈补充。这与题干“无法快速确定完整需求但核心需求清楚”完全对应。瀑布/V 要求需求明确；演化模型对核心需求清晰度要求不如增量模型。

2010-2011 期末

下面关于软件生命周期模型描述正确的是（ ）

- A. 软件生命周期是指从软件需求分析到上线运行的全过程
- B. V 模型和 W 模型可以解决需求不明确的软件项目
- C. RUP 本质上是喷泉模型
- D. 原型方法只能用于需求分析阶段

答案：C

解析：A 错：生命周期涵盖需求定义→开发→维护→废弃，不仅是到上线。B 错：教材 2.3.6 节明确“W 模型仍然只适合需求比较稳定的软件项目”，V/W 模型未解决需求不明确问题。C 对：教材 2.4.1 节指出“RUP 是融合喷泉模型和增量模型的一种综合生命周期模型”，故本质上是喷泉+增量，与喷泉模型最贴近。D 错：原型方法可支持生命周期多种阶段。

2011-2012 期末 RUP 模型定义与以下哪一个模型更加贴近（ ）

- A. 瀑布模型 B. 多个微型的瀑布模型 C. 演化模型 D. 螺旋模型

答案：B

解析：RUP 是迭代式增量过程模型，每个迭代周期内部按需求、分析设计、实现、测试等活动顺序展开，相当于一个微型瀑布；多次迭代构成整体生命周期。教材 2.3.1 节提到 Walker Royce（瀑布模型提出者 Winston Royce 之子）正是 RUP 主要创建者之一。RUP 在保留瀑布的阶段化优点同时引入迭代，故 B 更贴近。

2012-2013 期末 下面关于软件生命周期模型描述不正确的是（ ）

- A. 软件生命周期是指从软件需求分析到上线运行的全过程
- B. V 模型和 W 模型确定了软件测试的活动和测试对象
- C. 敏捷建模不是一个完整的方法论，但在应用传统的生命周期模型时，可以借鉴它的过程指导思想
- D. 原型方法不仅能应用于需求分析，也可以应用于设计和实现

答案：A

解析：教材 2.2 节给出定义：“软件生命周期是指软件产品从考虑其概念开始，到该软件产品不再使用为止的整个时期”——起点是“概念”而非“需求分析”，终点是“不再使用（废弃）”而非“上线运行”。B 对应 2.3.6 节；C 对应 2.4.2 节；D 对应 2.3.9 节。

2013-2014 期末 对于需求不明确的软件项目且需要使用面向对象方法，哪种软件生命周期模型最合适（ ）

- A. 瀑布模型 B. UP 模型 C. 喷泉模型 D. 演化模型

答案：B 或 C 均可

解析：教材 2.3.4 节明确“喷泉模型是以面向对象驱动的模式，主要用于描述面向对象的软件开发过程”，且支持阶段重叠与迭代，适合需求不明确的 OO 项目；2.4.1 节明确“RUP 是融合喷泉模型和增量模型的一种综合生命周期模型……用例为驱动，以架构为核心的迭代增量式开发模型”，同样支持 OO 与需求演变。瀑布（A）要求需求明确，演化（D）非面向对象专属，故 B、C 均可。

2018-2019 期末 以下哪项不属于增量模型的优点：_____

- A. 客户在第一次增量后即可使用系统的核心功能，增强对产品的信心
- B. 因为核心功能首先被开发，因此项目总体失败风险较小
- C. 最高优先级功能能够获得多次测试，从而保障系统重要功能
- D. 能够方便地将客户的需求映射到适当规模的增量上，加快软件开发的进程
- E. 所有增量都是在同一个体系结构指导下进行集成的，提供了系统的稳定性和可维护性

答案：D

解析：教材 2.3.3 节列出增量模型的 4 项优点（A、B、C、E）和 2 项缺点，其中第 1 项缺点正是“很难把客户的需求映射到适当规模的增量上”——即 D 项所述的“能够方便地将客户的需求映射到适当规模的增量上”与教材原文相反，故 D 不属于优点。

2023-2024 期末 (单选题) 本学期的课程作业采用的是以下哪一项生命周期模型 ()

A. 增量模型 B. 极限编程 C. 演化模型 D. UP 模型

答案: B (极限编程)

解析: 由本卷 id=421"请给出除了极限编程之外你认为最适合课程作业的 1 种生命周期模型"反推, 本学期课程作业采用的就是**极限编程 (XP)**。教材 2.4.3 节详细介绍 XP 的工作流程、角色、需求 (用户故事)、设计、编程 (结对) 与极限测试。

三、填空题 (共 3 题)

2023-2024 期末 敏捷开发中, 个体与交互重于过程与工具, 可工作的软件重于面面俱到的 (____), 客户合作重于合同谈判, 响应变化重于遵循计划。

答案: 文档

解析: 此为敏捷宣言 (Agile Manifesto, 2001) 四条价值观之一: "Working software over comprehensive documentation"。教材 2.4 节介绍以敏捷思想为代表的新型软件生命周期模型, XP (极限编程) 为典型代表。

2024-2025 期中 UP 模型的特点可以概括为以**用例为驱动**、() 为核心的应用 () 的生命周期模型。

答案: 架构; 迭代增量式 (即迭代和增量)

解析: 教材 2.4.1 节明确归纳 RUP 的三大特点: "综上所述, RUP 的特点可以概述为用例为驱动, 以架构为核心的迭代增量式开发模型"。故第一空填 "架构", 第二空填 "迭代增量 (式)"。

2024-2025 期中 极限编程的计划分为**交付计划**和 ()。

答案: 迭代计划 (Iteration Plan)

解析: 教材 2.4.3 节"XP 的工作流程"明确: XP 先给出初始的版本发布计划 (即交付 / Release Plan), 随后进入迭代开发过程, 其中包括迭代计划 (Iteration Plan) 的制订。故两类计划为 **交付 (发布) 计划** 与 **迭代计划**。

四、简答/问答题 (共 10 题)

2009-2010 期中 请阐述软件生命周期模型的作用, 并用图示方法给出演化模型及增量模型。

标准解答:

(1) 软件生命周期模型的作用:

1. 描述软件从需求定义到废弃的全部过程、活动和任务, 提供**结构框架**;
2. 作为开发人员按确定框架进行开发的**标准**;
3. 降低开发复杂度、提高透明度与可管理性;
4. 描述不同阶段产生的软件工件, 明确活动执行角色;
5. 便于项目管理人员对开发过程进行组织、调度、监控与质量保证。

(2) 演化模型图示 (两次开发):

第一次开发: 需求 → 分析 → 设计 → 实现 → 【试验性原型】
↓ 评估反馈
第二次开发: 需求修订 → 设计 → 实现 → 测试 → 【最终产品】

(3) 增量模型图示 (按优先级分批交付):

需求框架 → 体系结构定义
↓
[增量1=核心功能] → 设计→实现→测试→集成 → 交付 v1
[增量2=次优先] → ... → 交付 v2
[增量3=...] → ... → 交付 v3
↓
最终完整系统

2009-2010 期中 原型开发方法有哪三种类型? 原型方法的应用过程是什么? 原型方法可以应用于软件生命周期的哪些阶段?

标准解答:

(1) 3 类型: ①**探索型**——弄清用户要求, 探讨多种方案可行性 (针对需求模糊); ②**实验型**——考核技术方案是否合适、分析设计规格是否可靠 (大规模开发之前); ③**进化型**——适应需求变化, 逐步将原型进化成最终系统 (需求经常变动)。

(2) 应用过程 5 步: 快速分析 → 快速构造 → 用户使用 → 评价反馈 → 修改 (迭代)。

(3) 可应用的 5 种阶段范围:

1. 辅助或代替分析阶段;
2. 辅助设计阶段;
3. 代替分析与设计阶段;
4. 代替分析、设计和实现阶段;
5. **代替全部开发阶段 (典型演化模型)**。

2010-2011 期中 简述瀑布模型的定义及应用场景的优缺点。

标准解答:

定义 (教材 2.3.1): 瀑布模型规定软件生命周期中的基本工程活动, 并规定它们自上而下、相互衔接的固定次序, 如同瀑布流水逐级下落。每个阶段以上一阶段的输出 (评审通过的文档) 为输入, 本阶段产出软件工件并经评审后才能进入下一阶段; 若评审未通过则返回前一阶段返工。

应用场景: 需求明确的中小型项目, 特别是在需求稳定、技术成熟、有明确里程碑文档要求的项目中 (如军工、嵌入式、合规性要求高的系统)。

优点: ①阶段划分降低复杂度, 提高透明度, 便于工程化管理; ②推迟实现, 强调编码前必须完成分析与设计; ③以阶段评审和文档控制为手段指导整个开发过程, 保证产品质量。

缺点: ①缺乏灵活性, **无法解决需求不明确或不准确** (最突出缺点); ②风险控制能力弱 (成品只有测试后可见); ③文档驱动, 文档过多会增加工作量; 以文档完成度评估进度会产生错误结论; ④工期长, 初始成功率较低。

2010-2011 期末 请解释软件生命周期模型与本学期所学习的结构化与面向对象两种方法的关系。(5 分)

标准解答:

软件生命周期模型是描述软件从需求定义直至废弃所跨越的全部过程、活动和任务的结构框架, 规定了"**何时做什么**"。**开发方法** (结构化/面向对象) 则规定"**在每个阶段如何做**"——具体的建模与表示体系。两者是 **过程框架** 与 **方法学** 的正交关系: 同一个生命周期模型可分别结合结构化或面向对象方法使用。

与结构化方法的关系: ①以数据流图、数据字典、ER 图为分析工具, 以功能结构图、程序流程图设计工具; ②与瀑布模型天然契合——分析→设计→编码各阶段串行展开; ③也可在演化、增量、螺旋等模型下使用。

与面向对象方法的关系: ①以 UML 用例图、领域模型、顺序图分析工具; ②与喷泉模型、迭代增量模型、RUP 紧密契合——喷泉模型本就是"以面向对象驱动驱动模型"; RUP "用例为驱动, 以架构为核心的迭代增量式开发模型", 与 OOA/OOD 配合最佳; ③OOA 与 OOD 采用一致表示法, 使迭代增量模型能够高效运行。

综合: 生命周期模型提供"过程骨架", 方法学提供"建模与变换技术"。**结构化方法更适合需求稳定的瀑布式过程, 面向对象方法更适合迭代增量的喷泉/RUP 过程。**

2010-2011 期末 在本课程的实验项目中，你们经历了团队组织与开发管理的过程，甚至需求的变化，并应用面向对象技术进行软件的设计与实现。请结合该实验项目，回答以下问题（5 分）：

(1) 采用的是哪一种生命周期模型？后期的需求变更符合哪一种模型？

(2) 领域模型的类图与设计模型的类图有什么不同？

标准解答：

(1) 课程实验通常采用 **迭代增量模型 / 演化模型 / RUP**（用例为驱动、以架构为核心的迭代增量开发），先完成核心功能，再逐步扩展。若团队按需求分析→概要设计→详细设计→编码→测试的严格顺序一次性交付，则为 **瀑布模型**；若分版本交付，则为 **增量模型**。后期的需求变更更符合**演化模型 / 喷泉模型 / RUP**（迭代增量式）——这三类模型都明确支持需求变化。其中喷泉模型"以面向对象驱动……各阶段相互重叠和多次反复"，与本课程"应用面向对象技术且需求变动"的情境最契合。

(2) 领域模型类图 vs 设计模型类图：

维度	领域模型类图（分析）	设计模型类图（设计）
目的	抽象业务世界	描述软件系统内部结构，作为编码依据
类性质	概念类 （不是软件对象）	软件类 （可直接映射为源代码类）
包含内容	类名、属性、关联， 一般不含方法	含 操作/方法 、可见性、依赖等
抽象层次	高层、与实现无关	接近实现、考虑语言框架
来源	名词短语分析法识别	领域模型转换 + 设计新增（控制器、工厂、持久层等）

2013-2014 期末 请列举出软件生命周期中贯穿的软件工程过程的六个基本活动（4 分）

答案（6 个基本活动）：

1. **制定计划**（可行性分析、资源/成本/进度估计）
2. **需求分析和定义**（编写需求规格说明书）
3. **软件设计**（概要设计 + 详细设计）
4. **程序编写**（编码）
5. **软件测试**（单元/组装/有效性测试）
6. **运行 / 维护**（改正性/适应性/完善性维护）

教材 2.2.2 节明确："整个软件生命周期可以归纳出贯穿于软件工程的 6 个基本活动。"对应戴明环 PDCA。

2015-2016 期中 请解释"推迟了软件实现"是瀑布模型的一大优点。

答案：

"推迟"特指软件编码（实现）之前执行了瀑布模型中的**需求分析**和**软件设计**等活动，提高了软件质量。

详细解释：教材 2.3.1 节列举瀑布模型三大优点，其中第 (2) 条即"推迟软件实现"。其含义是：瀑布模型强调在编码（软件实现）之前，必须先完成需求分析和设计工作，**避免开发人员急于编写代码而忽视分析与设计**，最终导致代码频繁改动、结构混乱、质量下降。早期小作坊式开发正是由于直接编码，导致后期反复修改、维护成本极高。

2021-2022 期末 请说明软件需求分析阶段获取明确需求的难点有哪些？请结合软件生命周期模型说明可以有应对的方法？

标准解答：

获取明确需求的难点：

1. **用户说不清需求**：用户明白要什么，但表达能力有限；
2. **用户与开发人员沟通障碍**：知识背景差异大，存在理解偏差和二义性；
3. **需求本身动态变化**：业务环境、用户认知随时间变化；
4. 用户自身不确定（"想要什么"和"不要什么"分不清）；
5. 多个利益相关方需求冲突；
6. 分析人员缺乏领域知识；
7. 自然语言描述的二义性。

结合软件生命周期模型的应对方法：

1. **瀑布模型** (2.3.1)：仅适用于需求明确稳定项目；严格评审固化需求，文档驱动。
2. **演化模型 / 原型方法** (2.3.2、2.3.9)：构造原型让用户试用反馈，迭代明确需求——最主要的方法。
3. **增量模型** (2.3.3)：先交付核心功能，后续增量补充需求。
4. **喷泉模型** (2.3.4)：各阶段重叠且多次反复，可补充遗漏需求。
5. **螺旋模型**：风险驱动，每轮迭代都进行需求风险评审，适合需求高度不确定的大型项目。
6. **敏捷开发** (XP、Scrum 等)：用户全程参与，短迭代持续交付并调整需求。

2023-2024 期末 请给出除了极限编程之外你认为最适合课程作业的 1 种生命周期模型，并给出理由。

标准解答（推荐 UP / RUP）：

理由：

1. **用例驱动 + 以架构为核心 + 迭代增量**——RUP 的核心特征，与面向对象的课程作业（OOA→OOD→OOP）天然契合；
2. 阶段+迭代结构清晰：4 阶段（初始/细化/构造/交付），每个阶段可多次迭代，便于按里程碑评分；
3. 风险驱动：每次迭代在细化阶段重点识别和处理风险，避免课程末期才暴露架构问题；
4. 完整工件体系：用例模型、领域模型、设计模型、实现模型、测试模型齐全，便于训练 OOA/OOD 全流程；
5. 可裁剪：课程作业可裁剪为轻量化过程。

备选答案：**增量模型**（适合按模块分阶段交付）或 **瀑布模型**（如果作业要求一次性交付完整文档）。但 UP/RUP 与面向对象训练目标最匹配。

2024-2025 期中 请解释极限编程中有几种测试。（4 分）

标准解答：

从 **测试主体与目的** 角度，XP 的测试主要分为 **2 大类**：

1. **单元测试（Unit Test，由开发人员编写）**
 - XP 提倡 **测试驱动开发（TDD）**：在写程序之前先写单元测试；
 - 开发人员将 US 细化为编码任务后，对应每个任务编写单元测试用例；
 - 每次整合后都运行单元测试；代码复核与修改时须运行单元测试；发现 Bug 时增加相应测试；
 - 单元测试是开发过程中持续运行的"安全网"。
2. **验收测试 / 功能测试（Acceptance Test，由用户编写）**
 - 用户为每个 User Story 指定验收测试（功能测试），并运行验收测试；
 - 每个迭代结束时，团队依据客户的验收测试用例进行验收测试；通过后发布新版本，未通过则修复后重新测试。

从 **测试层次** 角度，教材 2.4.3 节"极限测试"还列出：单元测试、整合测试、功能测试、负荷测试、系统测试等。所有这些测试是 XP 开发过程中最重要的文档之一，也是最终交付给用户的内容之一。

结论：XP 的核心是 **单元测试 + 验收测试** 两类（两类主体不同）；从测试层级看还涵盖整合、功能、负荷、系统等测试。

A4 双栏 Cheat Sheet（考点速查）

🔗 软件工程过程

- 4 活动：规格说明 / 开发 / 确认 / 演进
- PDCA = P 制定计划、D 需求/设计/编码、C 测试、A 维护
- 3 类型：工作流 / 数据流 / 角色-动作

🌊 瀑布（Royce 1970）

- 特征：自上而下，相互衔接，固定次序
- 每阶段 4 动作：上一输出→执行→产出→评审
- 优：降复杂 / 推迟实现 / 文档控制
- 缺：缺灵活性（最突出）/ 风险控制弱 / 文档驱动
- 适用：需求明确中小项目
- 关键词：do twice、初步程序设计、演化基础

🔄 演化模型

- 2 次开发：试验原型 → 满意产品
- 分类：探索式（与用户共同工作）/ 抛弃式（明确需求后弃）
- 优：处理需求不明确、降低风险
- 缺：结构差、难管理、丢失文档控制
- 适用：需求不清楚、小型、短周期

📈 增量（Mills 1980）

- 核心 = 瀑布 + 演化优点
- 按优先级分批，先核心后扩展
- 优 4 条：信心 / 风险低 / 高优先级多测 / 体系结构稳
- 缺 2 条：粒度难选 / 基本服务难定
- 关键字：核心需求清楚但细节不清

💧 喷泉（迭代模型）

- 2 特征：迭代性 + 无间隙性
- 面向对象驱动；阶段无次序、可并行
- 优：提效 / 缩短周期
- 缺：难管理
- 教材：RUP 本质就是喷泉

🌀 螺旋（Boehm 1988）

- **最大特点 = 风险驱动（不是成本！）**
- 4 象限：制定计划 → 风险分析 → 实施工程 → 客户评估
- 大型项目；吸收演化概念
- 局限：需丰富风险评估经验

V 模型 / W 模型

- V：Paul Rook，测试分 4 级对应开发阶段
- 单元↔详细、集成↔概要、系统↔需求、验收↔用户需求
- V 局限：保留瀑布缺点，未提抗风险
- W：Evolutif；双 V 同步；测试对象含需求/设计
- 仍只适合需求稳定

🧩 构件组装

- 本质 = 演化的、迭代的
- 5 阶段：需求/分析→体系结构→构件开发→构造→测试发布
- 关键：体系结构设计
- 缺：缺通用标准、过分依赖构件

⚡ RAD

- 增量型 + 极短周期（60~90 天）+ 构件组装 + 4GL
- 5 阶段：业务/数据/过程建模→应用生成→测试迭代
- 适用：MIS；模块化好；3 月内可完成
- 不适用：技术风险高、互操作高

🍃 原型方法

- 3 类型：探索型 / 实验型 / 进化型
- 2 策略：废弃（探索+实验）/ 追加（进化）
- 5 步：快速分析→构造→使用→反馈→修改
- 5 范围：代替分析 / 设计 / 分设 / 分设实 / 全部开发
- 局限：大型需先划分；运算密集不适用；批处理不适用

💠 RUP 三大特点（必背）

- **用例为驱动 + 体系结构为核心 + 迭代增量**
- 本质：喷泉（迭代）+ 增量
- 二维：横轴=4 阶段+迭代；纵轴=9 workflow
- 每次迭代 = 一个微型瀑布

💠 RUP 4 阶段 4 里程碑

- 初始 → 软件目标里程碑
- 细化 → **体系结构里程碑**（解决全部技术风险）
- 构造 → 运行能力里程碑（beta 版）
- 交付 → 产品发布里程碑
- 口诀：初软目，细体建，构运能，交产发

🔦 RUP 9 workflow

- 6 过程：业务建模/需求/分析设计/实现/测试/部署
- 3 支持：配置变更管理/项目管理/环境
- 最佳实践：2~4 周迭代；早期解决高风险；UML 建模

🚀 敏捷 4 价值观

- 个体与交互 > 过程与工具
- 可工作软件 > 面面俱到**文档**
- 客户合作 > 合同谈判
- 响应变化 > 遵循计划
- AM 不是完整方法论（高频判断题）

⚡ XP（Kent Beck 1996）

- 轻量级；2~12 人；需求频繁变化、高风险
- 4 准则：沟通/简单/反馈/勇气
- 需求 = **User Story（US，非用例）**
- 关键：配对编程 / TDD / 持续集成 / 重构
- 计划：交付（发布）+ 迭代
- 测试：单元测试（开发）+ 验收测试（用户）

⚠️ 易错点

- "螺旋最大特点"→ 风险驱动（非成本）
- "迭代模型"→ 喷泉（非增量）
- "只有一次原型开发"→ 瀑布（无迭代）
- "不是完整方法论"→ 敏捷 AM
- "用例描述"是 UML，XP 用 User Story
- 生命周期从"概念"到"不再使用"
- V/W 不能解决需求不明确
- RUP 二维 = 时间维 + 活动维
- 原型方法可与其他模型组合（√）

🔗 模型选择速查

情境	首选
需求明确稳定	瀑布/V/W
需求完全不明确	演化/探索原型
核心清楚细节模糊	增量
大型+需求多变	螺旋
OO+迭代	喷泉/RUP
需求频变小团队	敏捷/XP
MIS 系统	RAD

🚀 6 基本活动速记

计划 → 需求 → 设计 → 编码 → 测试 → 维护
P → D → D → D → C → A
维护 3 类：改正性 / 适应性 / 完善性